

Arquitectura de Software

Designing Applications with AI Agents:

Patterns, Prompts, and Architectural Decisions

Presenter

Danilo Domínguez



Patterns

Agentic Workflows



Prompts

System Instructions



Decisions

Architectural Choices

Sobre mí

Danilo Domínguez Pérez

Educación

- **Ing. en Sistemas**
UTP (Egresado)
- **Maestría CC**
Rochester Institute of Technology
- **Doctorado CC**
Iowa State University - Investigación en Ingeniería de Software/Análisis de Programas

Experiencia

- **Software Engineer**
Plan A Technologies (Mobile)
- **Profesor**
UTP (Tiempo parcial)
- **Miembro activo**
FlossPA

Contacto



@danilo04



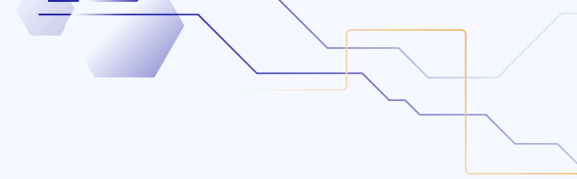
@danilo04



@danilo04.bsky.social



/in/danilo-dominguez-perez



Contenido

01

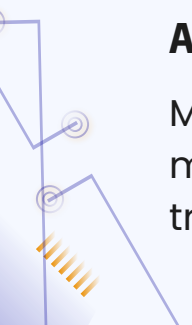
Arquitectura de Software

Definición, ley del trade-off y rol del arquitecto.

03

Agentic Systems

Modelos, prompts, tools, memoria, patrones y trade-offs.



02

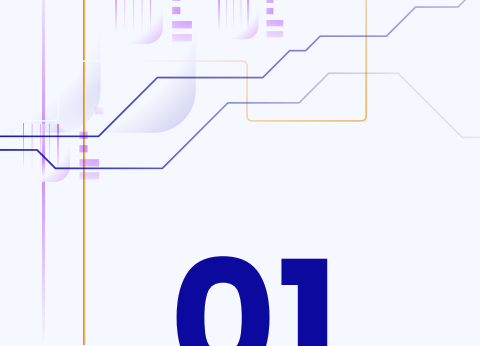
Componentes de un Proyecto

Requerimientos, restricciones y atributos de calidad.

04

Impacto en el Desarrollo

Cómo cambia el día a día y por qué el arquitecto importa más.



01 Arquitectura de Software

Decisiones estructurales que sostienen el producto.



¿Qué es arquitectura de software?

Conjunto de **decisiones estructurales** —costosas de cambiar— que definen cómo un sistema cumple sus requerimientos, restricciones y atributos de calidad.

> INSIGHT CLAVE

"La arquitectura no es un diagrama: es el razonamiento detrás de las decisiones y los trade-offs asumidos."

Estructura

Componentes, límites, interfaces y dependencias.

Razonamiento

Por qué se eligió X sobre Y: criterios, contexto y trade-offs documentados.



The first rule of software architecture
is that everything in software architecture
is a trade-off.

–Mark Richards & Neal Ford



Primera Ley de la Arquitectura de Software



No existe la bala de plata

Toda decisión deja algo atrás.



Depende del contexto

Lo óptimo hoy puede ser técnico mañana.



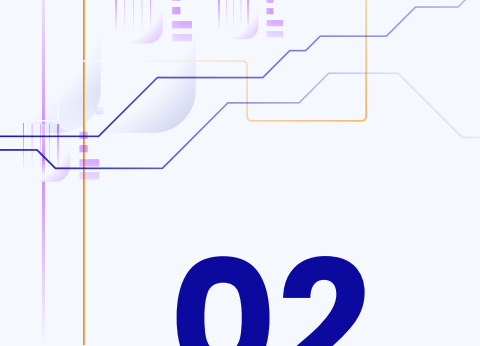
Explicita los trade-offs

Documenta qué ganas y qué cedes.

¿Qué hace un arquitecto de software?



El arquitecto sintetiza inputs de stakeholders, comunidad y restricciones técnicas en una arquitectura.



02 Componentes de un Proyecto

Los inputs que modelan la arquitectura.



Los 4 inputs de toda arquitectura



Requerimientos Funcionales

Qué debe hacer el sistema.



Restricciones de Negocio

Reglas, presupuestos, plazos, compliance.



Restricciones Técnicas

Stack, integraciones, infra, organización.



Atributos de Calidad

Los requisitos no funcionales que gobiernan el sistema.

Requerimientos Funcionales

Qué debe hacer el sistema.

- Features, flujos y casos de uso.
- Comportamiento observable (user stories, specs, BDD).
- Son el “qué”. ¿Qué problema resolvemos?
- Por sí solos no definen la arquitectura, pero establecen el alcance.



EJEMPLO

- Registrar usuarios
- Procesar pagos
- Generar reportes en PDF
- Notificar vía email
- Buscar productos por texto

Restricciones de Negocio

Lo que el negocio impone antes de decidir tecnología.

- Presupuesto y modelo de costos.
- Time-to-market y plazos contractuales.
- Regulación y compliance (GDPR, HIPAA, PCI...).
- Políticas internas, licencias y vendor lock-in.
- Mercado objetivo y geografías.



IMPACTO

El negocio te dice dónde NO puedes ir. Define el espacio de soluciones posibles y descarta opciones técnicamente válidas pero inviables.

Restricciones Técnicas

El ecosistema tecnológico en el que vive el sistema.

- Stack existente y deuda técnica.
- Integraciones obligatorias (ERP, CRM, legacy, APIs).
- Infraestructura y cloud provider.
- Conocimiento y tamaño del equipo (Conway's Law).
- SLAs de terceros y límites operativos.



TIP

El costo real de una decisión técnica incluye operarla, observarla, evolucionarla y contratar gente capaz de mantenerla durante años.

Atributos de Calidad

Los requisitos no funcionales que gobiernan cómo se comporta el sistema.

■ Performance

■ Scalability

■ Reliability

■ Availability

■ Security

■ Observability

■ Maintainability

■ Testability

■ Deployability

■ Portability

■ Usability

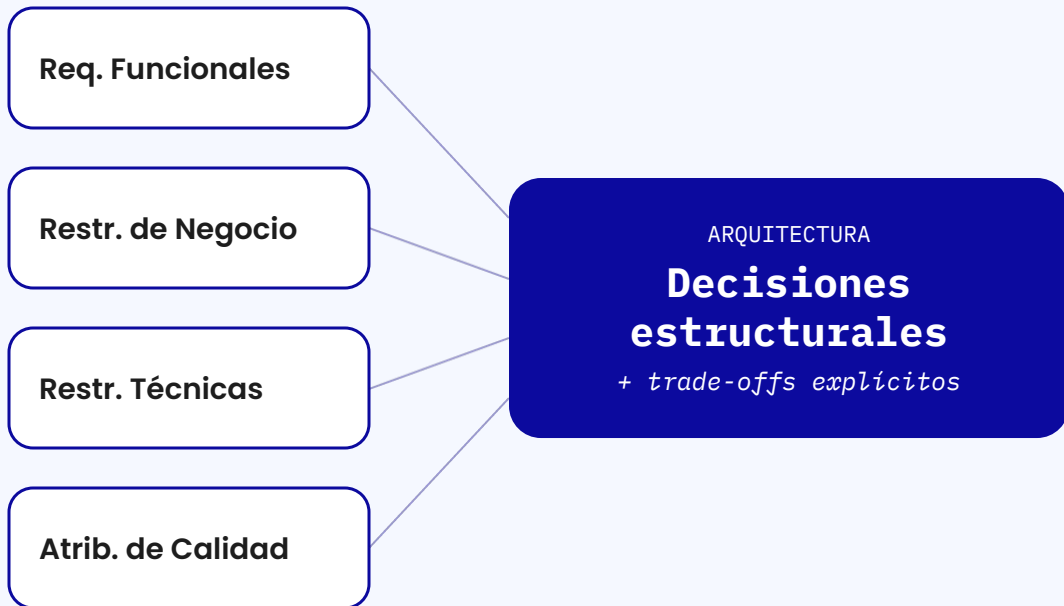
■ Cost-efficiency



RELEVANCIA

Son conflictivos entre sí. **Priorizarlos es la decisión arquitectónica más importante.**

¿Qué decide la arquitectura?



CLAVE

Los atributos de calidad y las restricciones son los que **más afectan** la arquitectura.

Lo funcional se puede implementar de muchas formas; los **requisitos no funcionales** descartan opciones.



03

Agentic Systems

Cuando el sistema razona, decide y actúa.



Agente vs No-Agente



Agente

Sistema autónomo que razona y actúa.

- Analiza datos de forma autónoma.
- Interpreta el entorno.
- Decide en base a contexto.
- Planifica pasos múltiples.
- Aprende de la retroalimentación.



No-Agente

Script o workflow disfrazado.

- Ejecuta pasos determinísticos.
- Flujo fijo sin razonamiento.
- Sin adaptación ante lo nuevo.
- Chatbots de reglas / árboles.
- RPA tradicional (UiPath, Zapier).

¿Qué hace posibles a los agentes hoy?

LLMs Frontier

GPT-5.5 • Claude Opus 4.6 • Llama 3 • GLM-5.2 • DeepSeek V4

Modelos capaces de razonar, usar herramientas y planificar pasos múltiples.



NLU

Comprende lenguaje natural ambiguo.



Contexto

Coherencia en interacciones largas.



Tool use

Invoca funciones para actuar.



Planning

Descompone metas en pasos multistep.



Unstructured

Emails, docs, logs, reportes.



Multimodal

Texto, imagen, audio, video.

Los LLMs dejaron de “completar texto” para “tomar decisiones ejecutables”.

De pipelines ML artesanales a agentes pre-entrenados

Lo que antes exigía meses de ingeniería ML hoy viene listo en el modelo.

// ANTES · CADA “AI” ERA UN PROYECTO

Pipeline ML artesanal

- 01 Recolectar datasets
- 02 Limpiar y etiquetar
- 03 Feature engineering
- 04 Entrenar y tunear
- 05 Validar y re-entrenar
- 06 Deploy + monitorear drift

Semanas a meses · Especialistas · Caro y frágil

// HOY · LLMs FRONTIER

Modelos pre-entrenados

GPT-5.5 - Claude 4.6 - Llama 3 - GLM 5.2

- Contexto largo
- Tool use
- Datos no estructurados
- Ambiguity solver
- Multi-step planning
- Multimodal core

De proyecto ML a integración de API

Los LLMs democratizan capacidades que antes exigían datasets propios, entrenamiento y un equipo de ML.

¿De verdad necesitas un agente?

Antes de construir un agente, evalúa 4 factores:

Variabilidad de inputs

¿Predecibles o abiertos?

Complejidad del razonamiento

¿Lineal o multi-paso con ambigüedad?

Performance / compliance

¿Latencia crítica?
¿Auditabilidad?

Mantenimiento continuo

¿Puedes operar y evaluar?

USA // 01

Código tradicional

Inputs predecibles y outputs enumerables.

USA // 02

Workflow determinístico

Bifurcaciones finitas y reglas claras.

USA // 03

Chatbot / RAG

Q&A sobre contenido, sin decisiones externas.

USA // 04

Agente autónomo

Inputs novedosos + multi-paso + aprendizaje.

Un agente solo se justifica cuando lo demás fracasa.

Código tradicional • Workflow • Agente

Un agente es la opción más costosa. Asegúrate de que el problema realmente lo requiere.

	Código tradicional	Workflow	Agente autónomo
Estructura de entrada	Totalmente previsible	Mayormente previsible, bifurcaciones finitas	Altamente no estructurada / novedosa
Explicabilidad	Transparencia total	Auditable rama por rama	Caja negra: requiere herramientas adicionales
Latencia	Ultra baja	Moderada	Mayor (inferencia LLM)
Adaptabilidad	Ninguna	Limitada	Alta (aprende del feedback)

Atributos de calidad para Agentic Systems



Scalability

Distribución, autoescalado y algoritmos paralelizables. Ej.: un agente que procesa 10 tickets/min debe sobrevivir a picos de 1.000 sin colapsar.



Modularity

Tools, prompts y memoria como módulos reemplazables. Evolucionas partes sin reescribir el agente.



Continuous learning

In-context learning + feedback de usuarios. Sin loops de feedback, el agente repite errores.



Resilience

Manejo de errores, timeouts, ataques y condiciones inesperadas. Degradación controlada.

Los 4 componentes de un agente



Model (LLM)

Razona y decide. Es el cerebro que analiza datos, interpreta el entorno y planifica pasos múltiples.



System Prompt

Instrucciones maestras. Define el rol, objetivos, estilo de interacción y los guardrails técnicos rígidos.



Tools

Acciones en el mundo. Building blocks funcionales para interactuar con APIs, DBs y otros servicios externos.



Memory

Contexto y aprendizaje. Gestión de información inmediata (short-term) y conocimiento persistente (long-term).

Decisiones Arquitectónicas y Componentes

Model (LLM)

Elegir modelo: 4 dimensiones de decisión

El modelo define cómo el agente interpreta inputs, genera outputs y se adapta al entorno. Modela performance, latencia y costo.

// 01 Task Complexity

Open-ended y ambiguo

- **Frontier:** GPT-5, Claude Opus 4.1 para razonamiento complejo y creatividad.
- **Small:** Phi-4, ModernBERT para tareas repetitivas y bien definidas.

// 02 Modality

Texto · imagen · audio · video

¿El input es solo texto o también multimodal (documentos, audio, video)?
Los modelos frontier actuales ya son nativamente multimodales.

// 03 Openness & Custom

API propietaria vs open weights

- **Open:** Llama, DeepSeek, GLM para control, compliance y fine-tune.
- **Custom:** Entrenamiento especializado donde la precisión de dominio es crítica.

// 04 Cost & Latency

La decisión operativa

- **Grande:** Cloud, mayor latencia y costo.
- **Chico:** Local, rápido y económico.
- **Híbrido:** Enrutamiento dinámico (routing) según complejidad.

Decide por dimensión, no por marketing. Tus evals + tu caso de uso deciden el ganador.

Frontier vs small: quién para qué caso

La dimensión más importante del framework. Casi siempre la respuesta final es «ambos».

// FRONTIER · LARGE FOUNDATION

Open-ended · razonamiento complejo

GPT-5 · Claude Opus 4.1 · Gemini Ultra · DeepSeek V3

- + Generalización, matiz, creatividad.
- + Tool use sofisticado y multi-step.
- + Maneja ambigüedad y contexto extendido.
- Alto costo por token, cloud obligatorio.
- Mayor latencia por llamada.

// CASOS DE USO

Personal assistants · research agents · enterprise systems con queries impredecibles.

// SMALL · DISTILLED · EFFICIENT

Estructurado · rápido · local

Phi-4 · ModernBERT · Gemma 2 · Llama 3.1 8B

- + Hardware local (GPU consumer).
- + Baja latencia y costo predecible.
- + Offline, privacidad y compliance.
- Menos flexibilidad ante lo novedoso.
- Fine-tune a veces necesario.

// CASOS DE USO

Customer support guiado · information retrieval · data labeling · clasificación y extracción.

Cuando el presupuesto o la latencia aprietan, el modelo chico gana por practicidad.

Open weight: MMLU vs required hardware

What I can self-host. Source: HELM (Stanford), MMLU as a general proxy.

Model	Vendor	MMLU	Params (B)	VRAM (GB)	Hardware
Llama 3.1 Instruct Turbo	Meta	56.1	8	20	RTX 3090
Gemma 2	Google	72.1	9	22.5	RTX 3090
NeMo	Mistral	65.3	12	24	RTX 3090
Phi-3	Microsoft	77.5	14.7	29.4	A100
Qwen 1.5	Alibaba	74.4	32	60.1	A100
Llama 3	Meta	79.3	70	160	4 x A100

MMLU is a starting point, not a verdict. Your evals in your domain are the real call.

Frontier: performance vs relative price

Relative price per 1M tokens (baseline: Llama 3.1 Turbo 405B = 1×).

Model	Vendor	MMLU	Input (x)	Output (x)
GPT-5.5	OpenAI	92.0	5	30
Claude Opus 4.6	Anthropic	91.3	5	25
DeepSeek-v4	DeepSeek	90.1	1.74	3.48
Llama 3.1 Instruct Turbo 405B	Meta	84.5	1	1

You can use: <https://benchlm.ai/compare/claude-opus-4-6-vs-gpt-5-5>

A decorative graphic in the top-left corner featuring a network of thin, colored lines (blue, orange, purple) and small dots, resembling a circuit board or data network.

Decisiones Arquitectónicas y Componentes

System Prompt

A decorative graphic in the bottom-right corner featuring a network of thin, colored lines (blue, orange, purple) and small dots, resembling a circuit board or data network.

System Prompt – 6 principios claves

El system prompt es código: versionado, probado y medido sobre las evals.

// 01

Clear Role Definition

"You are v0, Vercel's assistant."
Anchors behavior.

// 02

Structured Organization

Headers, <thinking> tags,
Markdown lists.

// 03

Explicit Tool Integration

When to call each tool,
I/O schema, errors.

// 04

Chain of Thought

Think before acting.
Mandatory "thinking" phase.

// 05

Environment Awareness

OS, cwd, available shell.
Agent grounding.

// 06

Domain Constraints

"Use Tailwind", "no package.json".
Hard technical rules.

Lessons from agents in production

Vercel v0

MDX COMPONENTS AS TOOLS

Instead of API calls, emits tags like `<CodeProject/>` or `<QuickEdit/>` to trigger UI changes. "No Apology" policy.

same.new

STRICT TOOL ETIQUETTE

Forbids mentioning tool names to the user. Requires explaining why a tool is being invoked before doing so.

Manus

EXPLICIT PLAN-ACT-OBSERVE LOOP

Mandatory cycle: gather information, process, reflect on the result before the next step.

Bolt.new

ENVIRONMENT CONSTRAINTS

Details what the WebContainer can and cannot do: "no Python", available commands, explicit limits.



Decisiones Arquitectónicas y Componentes

Tools (Herramientas)



Tools – las manos del agente

Los componentes básicos funcionales. Diseño modular → actualización sin necesidad de reescribir el agente.



Local tools

AGENT-INTERNAL LOGIC

Cálculos, reglas, base de datos local y funciones puras que operan directamente dentro del agente.



API-based tools

EXTERNAL SERVICES

Real-time CRM, payments, weather y REST/GraphQL integraciones third-party.



MCP

CONTEXT PROTOCOL

Model Context Protocol: un esquema abierto estándar para inyectar contexto, memoria y estado sin problemas.



BEST PRACTICES

Cada herramienta es un risk surface: validar entradas, autorizar permisos, auditar llamadas y delimitar el alcance. Diseñar módulos reemplazables, no estructuras monolíticas.

Definir una tool: LangChain + @tool

La tool es código normal. El agente obtiene el schema automáticamente desde la firma + docstring.

```
# tools/stock_price.py

from langchain_core.tools import tool
from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage

@tool
def get_stock_price(ticker: str) -> float:
    """Get the stock price for a given ticker."""
    url = f"https://api.example.com/stocks/{ticker}"
    response = requests.get(url)
    if response.status_code == 200:
        return response.json()["price"]
    raise ValueError(f"Failed to fetch {ticker}")
```

01 • DECORATOR

@tool

Transforma una función Python en una tool que el LLM puede invocar.

02 • SCHEMA

Type hints

ticker: str → float define el contrato que ve el modelo.

03 • INTENT

Docstring

El LLM la lee para decidir CUÁNDO llamar la tool.

Tool-calling loop: percibe → razona → actúa

Éste es el ciclo mínimo de un agente: el LLM elige la tool, nosotros la ejecutamos, él observa el resultado.

```
# agents/stock_agent.py
llm = ChatOpenAI(model_name="gpt-4o", temperature=0)
llm_with_tools = llm.bind_tools([get_stock_price])

messages = [HumanMessage("What is the stock price of Apple?")]

ai_msg = llm_with_tools.invoke(messages)
messages.append(ai_msg)

for tool_call in ai_msg.tool_calls:
    tool_msg = get_stock_price.invoke(tool_call)
    messages.append(tool_msg)

final_response = llm_with_tools.invoke(messages)
print(final_response.content)
```

01 • BIND

bind_tools({...})

Registra las tools. El modelo sabe qué puede llamar y con qué schema.

02 • DECIDE

ai_msg.tool_calls

El modelo decide QUÉ tool invocar y con qué argumentos. No ejecuta.

03 • LOOP

Percibe → razona → actúa

Agregamos la observación y re-invocamos hasta la respuesta final.

Decisiones Arquitectónicas y Componentes

Memoria

Memory – contexto y aprendizaje

SHORT-TERM

Contexto inmediato de la tarea

- Rolling context window / scratchpad.
- Coherencia durante la sesión.
- Descarta lo viejo, retiene lo relevante.
- *Ejemplo: chatbot que recuerda tus últimas preguntas.*

LONG-TERM

Conocimiento persistente

- Bases de datos, knowledge graphs, vector DBs.
- Preferencias, historial, hechos aprendidos.
- Habilita personalización en el tiempo.
- *Ejemplo: agente que monitorea signos vitales por años.*



Indexación

Organizar para recuperar rápido: keywords, embeddings, metadatos.



Relevancia

Diferenciar lo útil de lo ruidoso. Scoring y filtros contextuales.

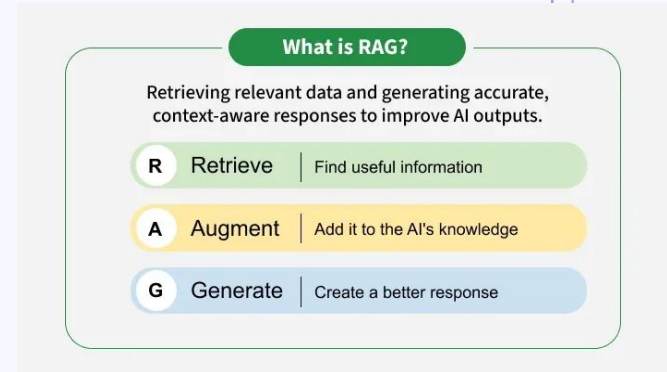


Olvido

Saber olvidar es un feature esencial: TTL, retención, privacidad.

RAG: Memoria a Largo Plazo

- **El Problema del Conocimiento** Los LLMs tienen una fecha de corte en su entrenamiento y carecen de acceso a los datos privados, dinámicos o confidenciales de tu organización.
- **Retrieval-Augmented Generation** Es el patrón arquitectónico de buscar información semánticamente relevante e inyectarla en el prompt antes de enviarlo al modelo.
- **Beneficios Clave** Mitiga drásticamente las alucinaciones, permite atribuir y citar fuentes exactas, y evita los altos costos de hacer fine-tuning constante al modelo.



De: <https://www.geeksforgeeks.org/nlp/what-is-retrieval-augmented-generation-rag/>

Arquitectura RAG: Fases Clave



1. Indexación

El proceso offline: los documentos corporativos se extraen, se limpian y se dividen en fragmentos lógicos (chunking). Luego, modelos de embedding convierten este texto a vectores numéricos que se almacenan en una Vector DB.



2. Recuperación

El proceso online: la consulta del usuario se transforma en un vector. El sistema realiza una búsqueda de similitud semántica en la base de datos para recuperar los fragmentos de contexto más relevantes.

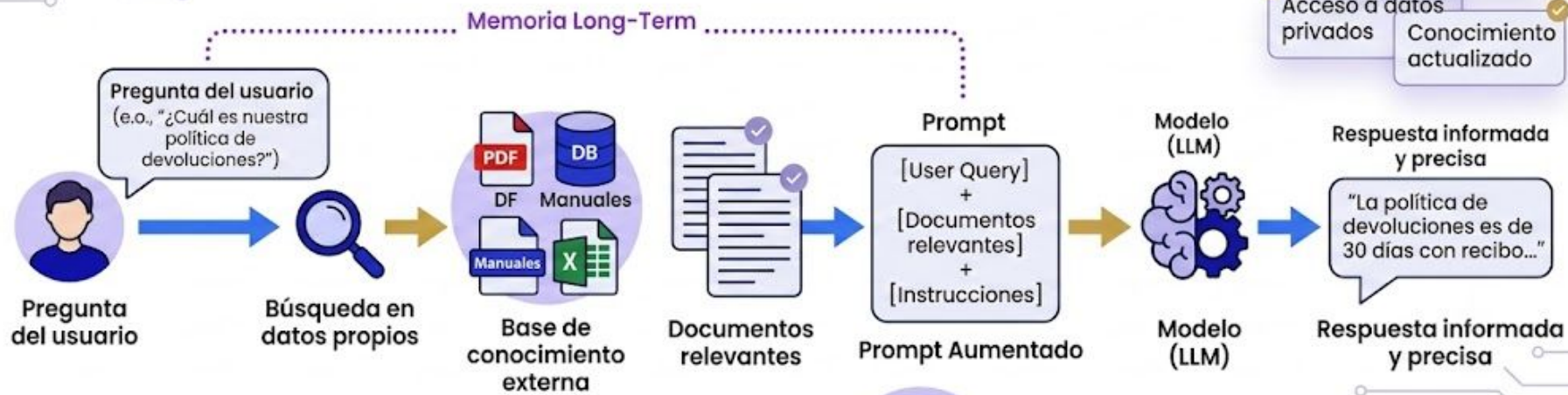


3. Generación

El orquestador aumenta el prompt original inyectando el contexto recuperado. El LLM recibe esta ventana de contexto enriquecida y genera una respuesta precisa, fundamentada estrictamente en los datos proporcionados.

¿Cómo funciona RAG?

¿Cómo funciona RAG? (Retrieval-Augmented Generation)





Decisiones Arquitectónicas y Componentes

Orquestración

Cómo se coordinan capacidades y agentes

La orquestación convierte capacidades aisladas en soluciones end-to-end: compone, planifica y supervisa secuencias de tools hacia un objetivo.



Single Agent

Un agente con muchas tools.
Simple y predecible.



Router / Supervisor

Un agente enruta hacia
especialistas dedicados.



Sequential Pipeline

Agentes en cadena de forma
sucesiva: salida → entrada.



Parallel Fan-out

Ramas de ejecución en paralelo
con un paso de reducción final.



Hierarchical

Estructura jerárquica: Manager →
sub-agentes. Anidable.



Graph / Stateful

Máquina de estados con flujos no
lineales, ciclos y condiciones
complejas.

Single-Agent vs Multi-Agent

Regla: empieza single-agent. Sube a multi-agent sólo cuando el dolor lo exija.

// SINGLE-AGENT

Un agente, muchas tools

VENTAJAS

- Simple de diseñar y operar.
- Sin costos de coordinación.
- Predecible y fácil de evaluar.

LÍMITES

- Se satura con tareas muy diversas.
- Diseño monolítico: difícil de evolucionar.

// MULTI-AGENT

Equipo de agentes especializados

VENTAJAS

- Especialización por dominio.
- Paralelismo real.
- Redundancia y resiliencia.

COSTOS

- Comunicación = más tokens, más costo.
- Coordinación compleja, más deuda operativa.



Decisiones Arquitectónicas y Componentes

Trade-offs (Compensaciones)



Los 4 trade-offs que definen tu agente

// PERFORMANCE

Speed ⇌ Accuracy

Respuesta rápida y aproximada, luego refinamiento más lento y preciso (estrategia híbrida).

// SCALABILITY

Concurrencia ⇌ Costo

GPU dinámico bajo demanda, autoscaling, horarios off-peak.

// RELIABILITY

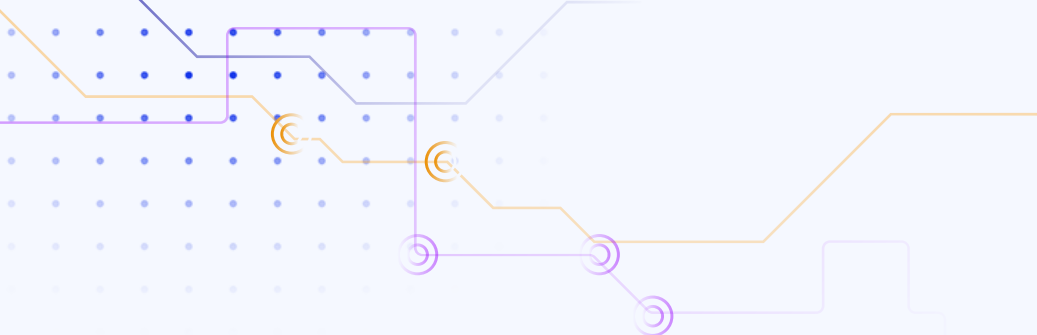
Consistencia ⇌ Complejidad

Fault tolerance, redundancia, monitoreo continuo y testing de edge cases.

// COST

Capacidad ⇌ Presupuesto

Modelos caros sólo para misión crítica. Routing dinámico para el resto.



Patrones de Diseño



Patrones de diseño que querrás conocer

// 01 ReAct

Reason + Act en loop: piensa → actúa → observa.

// 02 Plan-and-Execute

Primero planifica, luego ejecuta paso a paso.

// 03 RAG

Retrieval-augmented generation para grounding.

// 04 Reflection / Critic

El agente critica su propia salida antes de responder.

// 05 Multi-agent debate

Agentes contrastan posturas y deciden.

// 06 Tool-first

Siempre intenta usar tools antes de responder de memoria.

// 07 Guardrails

Input/output classifiers. Valida entradas y salidas.

// 08 Human-in-the-loop

Aprobación humana en pasos críticos.

Frameworks

// SEQUENTIAL APPS

LangChain

Diseñado para construir aplicaciones secuenciales y flujos lineales simples.

// CYCLIC GRAPHS

LangGraph

Evolución de LangChain diseñada para sistemas complejos con flujos cíclicos.

// KOTLIN-BASED

Koog

Framework basado en Kotlin con soporte para herramientas avanzadas y grafos complejos.

// MULTI-AGENT

AutoGen

Orquestación multi-agente potente, asignación dinámica de roles y mensajería flexible.

...y muchos otros frameworks en constante evolución.



Caso de Estudio: Bayer

Construyendo aplicaciones LLM confiables (Reliable LLMs)
para la ciencia de cultivos.

<https://martinfowler.com/articles/reliable-llm-bayer.html>

El Desafío de los Datos Preclínicos

01. INFORMACIÓN NO ESTRUCTURADA

Décadas de valioso conocimiento preclínico se encontraba atrapado dentro de reportes PDF no estructurados.

02. PÉRDIDA DE METADATOS

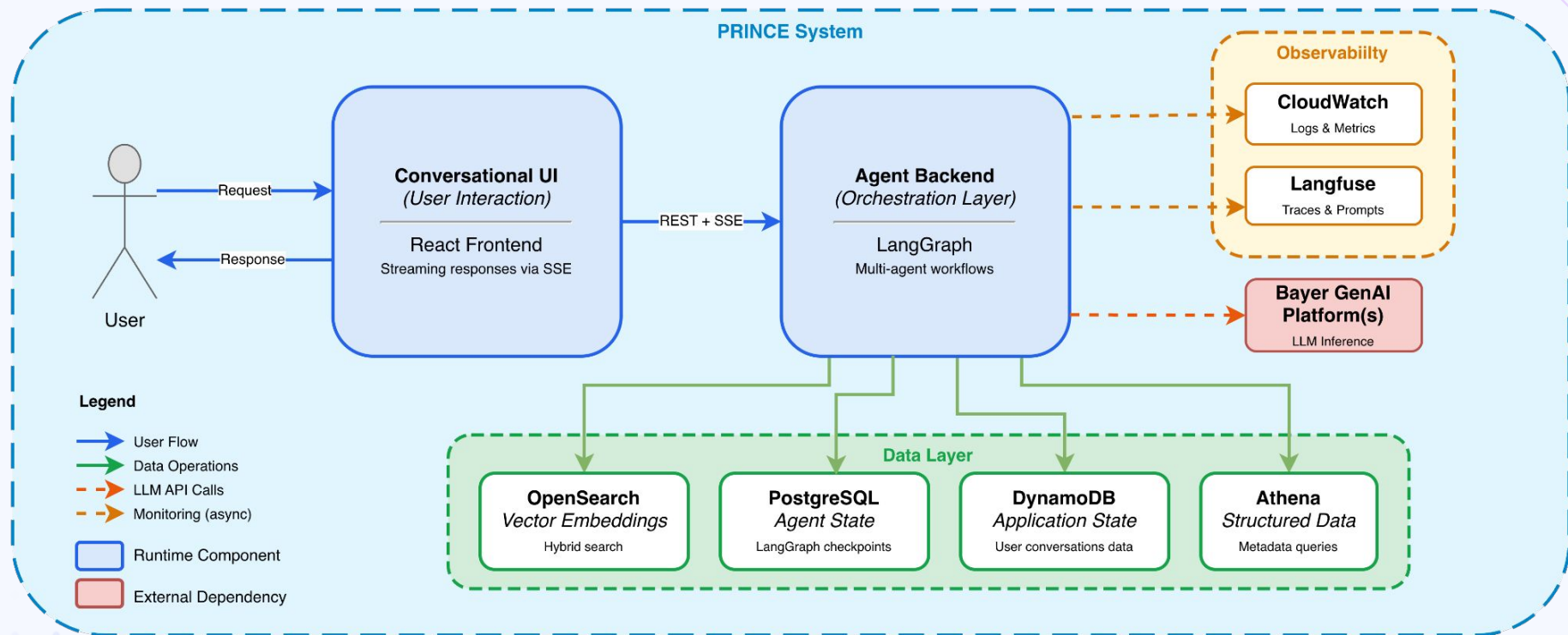
Debido a migraciones de sistemas a lo largo de los años, los metadatos estructurados asociados estaban incompletos, faltantes o incorrectos.

03. EXIGENCIA REGULATORIA

Los investigadores necesitaban un sistema capaz de recuperar información con precisión para redactar documentos regulatorios complejos.



Arquitectura



Atributos de Calidad

Reliability en Acción

Explainability

Transparencia Total

Dada la naturaleza iterativa del workflow, el sistema expone los **pasos intermedios**. El usuario puede ver el razonamiento, las queries formuladas y las herramientas utilizadas para llegar a la respuesta.

Factuality

Citas Robustas

Cada respuesta está acompañada de **citas directas**. Los usuarios pueden inspeccionar cualquier oración para ver el documento fuente, número de página exacto y el extracto de texto utilizado.

Resilience

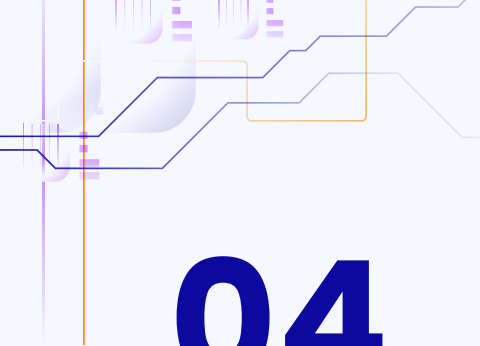
Continuidad de Servicio

Si un LLM específico falla o sufre una caída, el sistema implementa reintentos automáticos y **fallbacks a modelos alternativos** para asegurar que el investigador no pierda su flujo de trabajo.

RAG Arch.

Contexto Confinado

Uso exhaustivo de Retrieval-Augmented Generation. Se fuerza al modelo a responder **únicamente en base al contexto inyectado** desde la base de datos vectorial corporativa.



04

Impacto en el Desarrollo

Developers, equipos y el regreso del arquitecto.



Cómo cambia el día a día del developer

El developer deja de “**escribir código**” y empieza a “**dirigir sistemas que escriben código**”.

// AYER

- Escribir todo el código manualmente.
- Seguir patrones aprobados al pie de la letra.
- Foco en sintaxis, boilerplate, repetición.
- Revisiones humanas lentas.
- Iteración marcada por builds y releases.

// CON AGENTES

- Co-diseñar con agentes: **prompts como especificación.**
- Menor boilerplate, más decisiones de diseño.
- Nuevas skills: **evals, guardrails, observabilidad.**
- Tests y revisiones como guardrails vivos.
- Ciclos de feedback mucho más cortos.

Por qué el rol del arquitecto toma fuerza

Más decisiones, más difíciles

Modelos, prompts, tools, memoria, orquestación, costos: explota el espacio de diseño.

Trade-offs no triviales

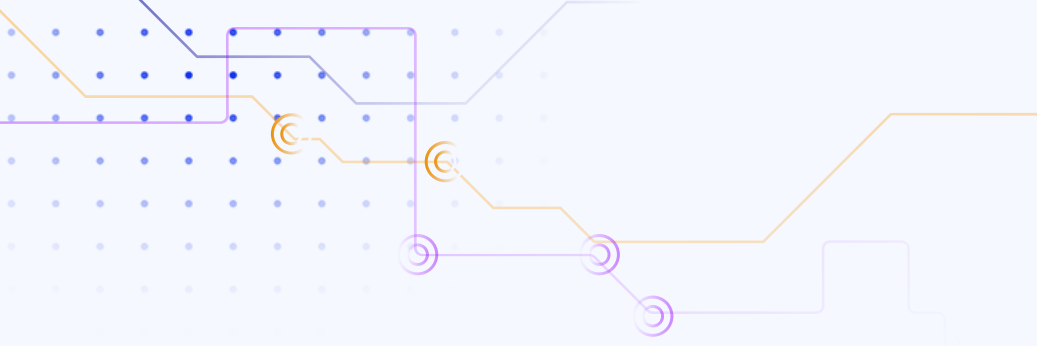
Calidad vs costo vs latencia vs seguridad. Alguien debe razonarlos explícitamente.

Riesgos nuevos

Prompt injection, hallucinations, fugas de datos, comportamiento no-determinístico.

La IA ejecuta, el humano decide

Los agentes aceleran el "cómo". El arquitecto protege el "qué" y el "por qué".



Claves



Claves para llevarte

// 01

Todo es un trade-off

Aplica igual para agentes que para sistemas tradicionales. Hazlos explícitos.

// 02

Los requisitos no funcionales mandan

Lo que decide arquitectura no son las features, son restricciones y atributos de calidad.

// 03

Los agentes son sistemas

Model + prompt + tools + memory. Aplica ingeniería, no magia.

// 04

El arquitecto protege el por qué

La IA acelera el cómo; el humano decide qué y por qué.

// SECCIÓN DE PREGUNTAS

¿Quién tiene la primera pregunta?

Escanea el código QR de la derecha para enviar tus comentarios, evaluar la presentación y acceder a los recursos del evento.



Designing Applications with AI Agents: Patterns, Prompts, and Architectural Decisions

[ESCANEAR CÓDIGO QR]